

International Islamic University Islamabad

Faculty of Engineering and Technology

Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 7: Implementation of Regression Techniques using scikit-learn Library

Name of Student: Bushra Nazir

Registration No.: 1071-F22

Date of Experiment: 7-4-2026

Submitted To: Engr.Asad

Implementation of Regression Techniques using scikit-learn Library

Objectives

- To understand the concept of linear regression and its application in supervised learning.
- To implement linear regression using the scikit-learn library for model training, prediction, and evaluation.
- To compare the results of regression implemented using scikit-learn with a basic implementation from scratch using NumPy.

Equipment Required

- Raspberry Pi or Computer with Python installed
- DHT11 or DHT22 Temperature and Humidity Sensor
- Jumper Wires
- Breadboard
- Resistors (as required for sensor connection)
- Internet Connection (for installing Python packages)
- IDE or Code Editor (Thonny, VS Code, Jupyter Notebook, etc.)

Introduction to scikit-learn

Scikit-learn is one of the most widely used and powerful open-source Python libraries for machine learning. It provides simple and efficient tools for data mining, data analysis, and building predictive models. Built on top of NumPy, SciPy, and matplotlib, scikit-learn supports a wide range of supervised and unsupervised learning algorithms such as regression, classification, clustering, and dimensionality reduction. Its user-friendly interface and consistent API make it ideal for both beginners and professionals working in the field of data science and machine learning.

Scikit-learn is extensively used in a variety of real-world applications. For instance, it is employed in healthcare to predict disease progression, in finance to forecast stock prices and detect fraud, and in e-commerce for customer segmentation and recommendation systems. Whether you're analyzing housing prices, diagnosing illnesses, or developing intelligent IoT systems on hardware like the Raspberry Pi, scikit-learn serves as a core tool in translating raw data into actionable insights.

Data Preprocessing using scikit-learn

Building on the capabilities introduced in scikit-learn, one of the most crucial steps before training any machine learning model is data preprocessing. Real-world data is rarely clean or structured; it often contains missing values, categorical variables, and inconsistent scales. Scikit-learn offers a comprehensive suite of tools that simplify these challenges and prepare data for efficient and effective learning. Preprocessing includes operations like handling missing values, encoding categorical features, feature scaling, and splitting the dataset into training and testing sets.

Before applying any machine learning algorithm, it's crucial to ensure that the dataset is clean, consistent, and properly formatted. Data preprocessing is the step that transforms raw data into a suitable format for training models. Scikit-learn, a powerful machine learning library in Python, provides simple and efficient tools for this task.

Preprocessing typically includes:

- **Handling missing values**
- **Encoding categorical variables**
- **Feature scaling**
- **Splitting data into training and test sets**

These steps are vital to improve model performance and ensure reliable results.

Installation of Library

Before installing scikit-learn, ensure that you have NumPy and SciPy installed. Once you have a working installation of NumPy and SciPy, **the easiest way to install scikit-learn is using pip:**

```
!pip install -U scikit-learn
```

Step-by-step Implementation

1. Import Required Libraries

Data preprocessing is a crucial step in machine learning that ensures your data is clean, formatted correctly, and ready for model training. The process begins by importing

essential libraries like pandas, numpy, and scikit-learn.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

2. Load a Sample Dataset

Data preprocessing is a crucial step in machine learning that ensures your data is clean, formatted correctly, and ready for model training. The process begins by importing essential libraries like pandas, numpy, and scikit-learn. You can either use a **sample dataset** (as shown in the previous example) or load an **external dataset**, typically stored in formats like CSV, Excel, or directly from databases or APIs. External datasets often contain missing values, inconsistent scaling, or categorical variables that need to be addressed.

```
data = {
    'Temperature': [22.4, 23.1, np.nan, 24.0, 23.8],
    'Humidity': [55, 58, 60, np.nan, 57],
    'Output': [0, 1, 0, 1, 1]
}

df = pd.DataFrame(data)
```

3. Separate Features and Target

separate the features (independent variables) from the target (dependent variable).

```
X = df[['Temperature', 'Humidity']]
y = df['Output']
```

4. Handle Missing Values

Handling missing values is an important task, and scikit-learn's SimpleImputer can be used to fill missing values with the mean of the respective column.

```
imputer = SimpleImputer(strategy='mean') # fill missing with column mean
X_imputed = imputer.fit_transform(X)
```

5. Feature Scaling

After addressing missing values, you use feature scaling (e.g., `StandardScaler`) to normalize the data, which helps machine learning algorithms perform better.

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_imputed)
```

Split the Dataset

Finally, the dataset is split into training and testing sets using `train_test_split` to ensure that the model is evaluated on data it hasn't been exposed to during training. These preprocessing steps, including the handling of real-world, external datasets, are essential to ensure that machine learning models are trained on clean, structured data.

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

Linear Regression using scikit-learn

Introduction to Regression:

Regression is a fundamental concept in machine learning and statistics, used to model the relationship between a dependent variable (also called the target or output) and one or more independent variables (also called predictors or features). The goal of regression is to predict the dependent variable's value based on the values of independent variables. The simplest and most widely used form of regression is **Linear Regression**.

In Linear Regression, the relationship between the dependent variable Y and the independent variable(s) X is assumed to be linear. The model attempts to fit a straight line (in the case of one feature) or a hyperplane (in the case of multiple features) to the data, minimizing the difference between the predicted and actual values. This is achieved through a method known as **least squares**, which minimizes the sum of the squared errors (the difference between the observed and predicted values). Mathematically, a simple linear regression model can be written as:

$$Y = \beta_0 + \beta_1 X + \epsilon$$

Where:

- Y is the dependent variable (output)
- X is the independent variable (input)
- β_0 is the intercept (the value of Y when $X=0$)
- β_1 is the slope (the change in Y for a one-unit change in X)
- ϵ represents the error term, which accounts for the difference between the predicted and actual values.

In real-world applications, multiple independent variables can be used in a **multiple linear regression** model:

Linear Regression in scikit-learn:

scikit-learn provides an easy-to-use implementation of linear regression. The **LinearRegression** class in scikit-learn helps fit a linear model to the data using the least squares approach.

To apply linear regression using scikit-learn, we typically:

1. Import the necessary libraries
2. Load and preprocess the dataset
3. Split the dataset into training and testing sets
4. Create an instance of the Linear Regression model
5. Train the model on the training data
6. Evaluate the model using testing data
7. Make predictions and assess the model's performance

The advantages of using scikit-learn for linear regression include simplicity, efficiency, and powerful tools for model evaluation.

Implementation of Linear Regression with direct tabular dataset

Let's consider a real-world scenario where we predict the house price based on factors like square footage and number of rooms. Here's a step-by-step guide and code to implement linear regression for this problem. We will assume a small dataset containing features like square footage and number of rooms, and the target variable will be the houseprice.

1. Import the necessary libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

2. Load the Dataset:

Let's assume we have the following sample dataset stored in a CSV file:

Square Footage	Rooms	House Price (\$)
1000	3	300,000
1500	4	400,000
1800	4	450,000
2000	5	500,000
2200	5	600,000

```

data = {
    'Square Footage': [1000, 1500, 1800, 2000, 2200],
    'Rooms': [3, 4, 4, 5, 5],
    'House Price': [300000, 400000, 450000, 500000, 600000]
}

# Convert to DataFrame
df = pd.DataFrame(data)

# Features (independent variables)
X = df[['Square Footage', 'Rooms']]

# Target (dependent variable)
y = df['House Price']

```

3. Split the Dataset

```

# Split data into training and testing sets (80% for training, 20% for testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=42)

```

4. Create and Train the Model:

```

# Create an instance of the LinearRegression model
model = LinearRegression()

# Train the model on the training data
model.fit(X_train, y_train)

```

5. Model Evaluation:

```

# Predict on the testing set
y_pred = model.predict(X_test)

# Evaluate the model's performance using Mean Squared Error (MSE) and R-squared
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared: {r2}")

```


6. Model Coefficients:

You can also check the coefficients (weights) of the features:

```
# Model coefficients
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```

7. Making Predictions:

Let's predict a new house:

```
# Predict price for a house with 1500 sq ft and 4 rooms
new_house = np.array([[1500, 4]])
predicted_price = model.predict(new_house)
print(f"Predicted House Price: ${predicted_price[0]:,.2f}")
```

Linear Regression using Data Set in CSV File

In this example, we will use the salary data concerning the experience of employees. In this dataset, we have two columns *YearsExperience* and *Salary*.

[The dataset file will be provided to you in class]

Step 1: Import the required python packages

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from pandas.core.common import random_state
from sklearn.linear_model import LinearRegression
```

Step 2: Load the dataset

Upload the dataset to your notebook and read it into the pandas dataframe.

```
# Get dataset
df= pd.read_csv('Salary_Data.csv')
df.head()
```

Output will be

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

Step 3: Data analysis

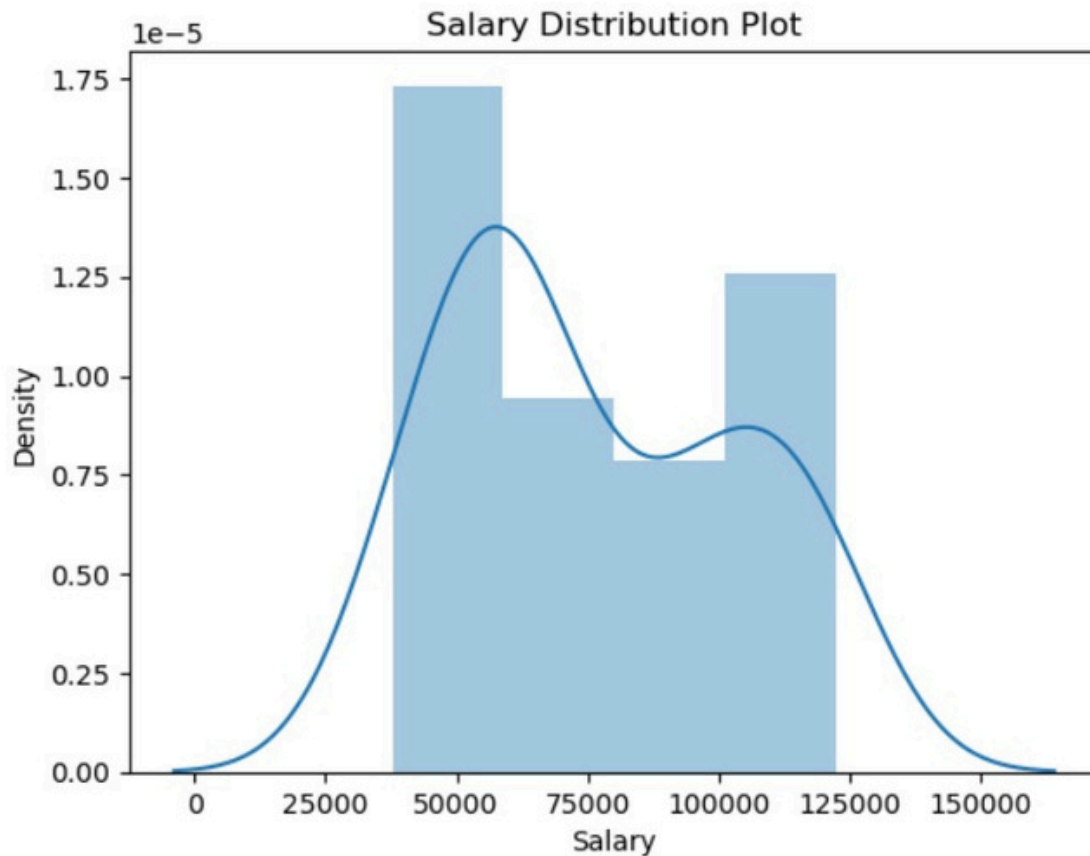
Now we will have some analysis of the data as below.

```
df.describe()
```

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

We can also find how the data is distributed visually using *Seaborn* distplot. A distplot or distribution plot shows the variation in the data distribution. It represents the data by combining a line with a histogram.

```
plt.title('Salary Distribution Plot')
sns.distplot(df['Salary'])
plt.show()
```



Next, we check the relationship between *Salary* and *Experience*.

```
# Relationship between Salary and Experience
plt.scatter(df['YearsExperience'], df['Salary'], color = 'red')
plt.title('Salary vs Experience')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.box(False)
plt.show()
```



It is clearly visible now, our data varies linearly. That means, that an individual receives more *Salary* as they gain *Experience*

Step 4: Split the dataset into dependent/independent variables

- *Experience (X)* is the independent variable.
- *Salary (y)* is dependent on experience

```
# Splitting variables
X = df.iloc[:, :1] # independent
y = df.iloc[:, 1:] # dependent
```

Step 5: Split data into Train/Test sets

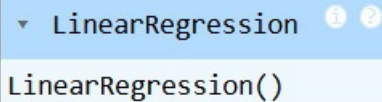
Further, split your data into training (80%) and test (20%) sets using `train_test_split`

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

Step 6: Train the regression model

Pass the *x_train* and *y_train* data into the regressor model by ***model.fit*** to train the model with our training data

```
model = LinearRegression()  
model.fit(X_train, y_train)
```



```
LinearRegression()
```

Step 7: Predict the result

Now the model has been trained and we can use it to predict any value of *y* (*Salary*) dependent on *x* (*Experience*) with the trained model using ***model.predict***.

```
y_pred_test = model.predict(X_test)    # predicted value of y_test
```

```
y_pred_train = model.predict(X_train)  # predicted value of y_train
```

Step 8: Plot the training and test results

It's time to test our predicted results by plotting graphs

- Plot training set data vs predictions

First we plot the result of training sets (*x_train*, *y_train*) with *x_train* and predicted value of *y_train* (*model.predict(x_train)*)

```
# Prediction on training set  
plt.scatter(X_train, y_train, color = 'lightcoral')  
plt.plot(X_train, y_pred_train, color = 'firebrick')  
plt.title('Salary vs Experience (Training Set)')  
plt.xlabel('Years of Experience')  
plt.ylabel('Salary')  
plt.legend(['X_train/Pred(y_test)', 'X_train/y_train'], title = 'Sal/Exp', loc='best', facecolor='white')  
plt.box(False)  
plt.show()
```



- Plot test set data vs predictions

Secondly, we plot the result of test sets (x_{test} , y_{test}) with x_{train} and predicted value of y_{train} ($model.predict(x_{train})$)

```
# Prediction on test set
plt.scatter(X_test, y_test, color = 'lightcoral')
plt.plot(X_train, y_pred_train, color = 'firebrick')
plt.title('Salary vs Experience (Test Set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend(['X_train/Pred(y_test)', 'X_train/y_train'], title = 'Sal/Exp', loc='best', facecolor='white')
plt.box(False)
plt.show()
```



In the end we can find c (*y-intercept*) and m (*slope/coefficient*) from the regressor model.

```
# Regressor coefficients and intercept  
print(f'Coefficient: {regressor.coef_}')  
print(f'Intercept: {regressor.intercept_}')
```

```
Coefficient: [[9312.57512673]]  
Intercept: [26780.09915063]
```


Lab Task

Implement a Motion-Triggered Data Logging System using PIR and OLED, where human motion detected by a PIR sensor triggers real-time data capture (e.g., timestamp, ambient conditions), and the motion status ("Motion Detected" / "No Motion") is displayed on an OLED screen. This system can serve as a trigger mechanism for collecting event-based data points later used for linear regression analysis (e.g., motion frequency vs. temperature).

Task:

Implement a Motion-Triggered Data Logging System using PIR and OLED, where human motion detected by a PIR sensor triggers real-time data capture (e.g., timestamp, ambient conditions), and the motion status (“Motion Detected” / “No Motion”) is displayed on an OLED screen. This system can serve as a trigger mechanism for collecting eventbased data points later used for linear regression analysis (e.g., motion frequency vs. temperature).

Code:

```
import time
import RPi.GPIO as GPIO
import board
import busio
import adafruit_ssd1306

PIR_PIN = 17

GPIO.setmode(GPIO.BCM)
GPIO.setup(PIR_PIN, GPIO.IN)

i2c = busio.I2C(board.SCL, board.SDA)
oled = adafruit_ssd1306.SSD1306_I2C(128,64,i2c)

while True:

    motion = GPIO.input(PIR_PIN)

    oled.fill(0)

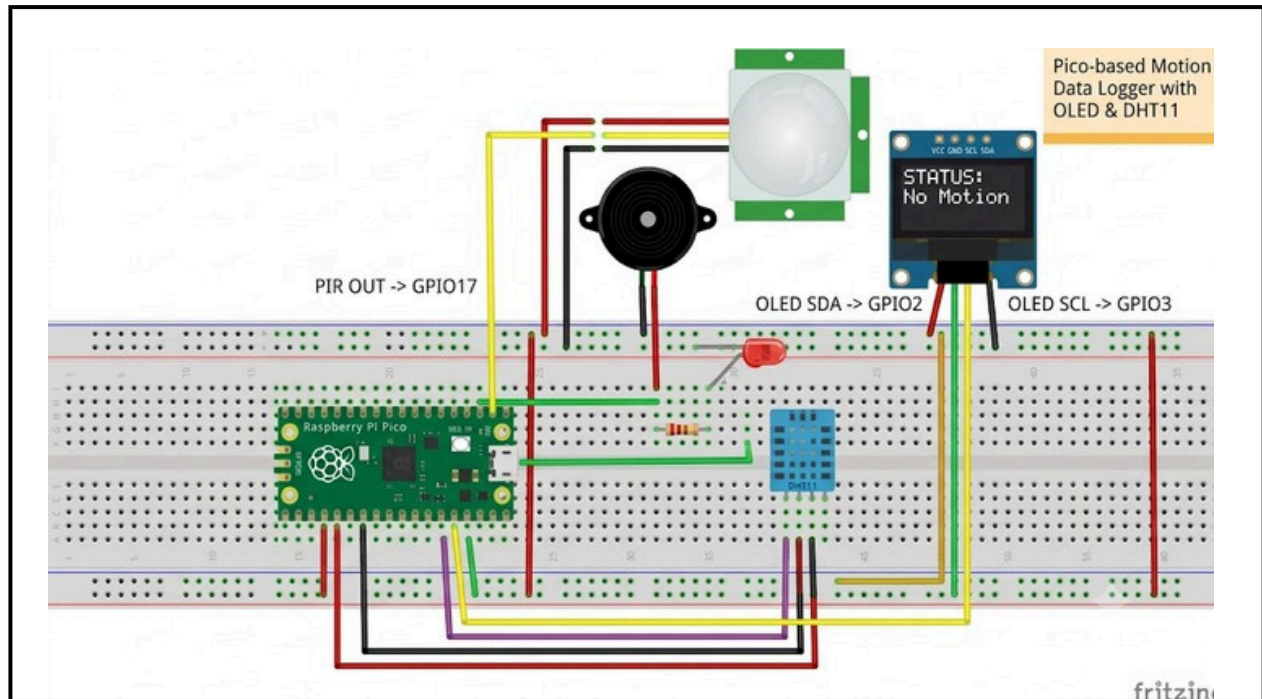
    if motion:
        print("Motion Detected")
        oled.text("Motion Detected",0,20,1)

    else:
        print("No Motion")
        oled.text("No Motion",0,20,1)

        oled.show()

        time.sleep(1)
```

Block Diagram:



International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 7

Implementation of Regression Techniques using scikitlearn Library

Name: Bushra Nazir Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
SUB-TOTAL (PT)		7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
SUB-TOTAL (CT)		1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
SUB-TOTAL (AT)		2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) :

10

Marks Obtained (PO + CO + AO) :

Instructor